MODULE 13 · DOMAIN 2 - ML MODEL AND FM DEVELOPMENT

Built-in algorithms

Organised by the shape of the problem, not by alphabet.

MODULE 13 of 43 DOMAIN 2 - Model Dev WEIGHT 24%

admin@sthithapragna.com

A built-in algorithm is an implementation you do not have to write, debug, or make fast.

Problem shape to algorithm, in four steps

01

Data type

Tabular, text, images, or a time series?

02

Labels?

Supervised if you have them, unsupervised if not.

03

Output

A number, a class, a group, a score, a sequence?

04

Name it

Usually only one or two algorithms remain.

ON THE EXAM

Four questions, asked in this order, cut eighteen algorithms down to two on almost every scenario.

Most commercial machine learning is **rows and columns**, and this is where it lives.

XGBoost

- **Gradient boosted trees.** Classification, regression, and ranking.
- **The strongest general performer on tabular data.** If a question says tabular, start here.
- Handles mixed numeric and categorical features, and missing values, without complaint.
- **Scale-invariant**, from module 05. Feature scaling changes nothing.
- Supports distributed training, and CSV or libsvm input. First column is the label.

> ON THE EXAM
>
> It wins because boosting builds each tree to correct the errors of the last, which suits messy real-world tabular data.

The XGBoost hyperparameters worth knowing

PARAMETER	WHAT IT CONTROLS
max_depth	Tree depth. Higher means more complex and more overfitting
eta	Learning rate. Lower is safer and needs more rounds
num_round	How many trees. Too many overfits
subsample	Fraction of rows per tree. Below 1 reduces overfitting
gamma	Minimum loss reduction to split further. Higher is more conservative
alpha and lambda	L1 and L2 regularisation on weights
scale_pos_weight	Weights the positive class. The imbalance lever from module 08

ON THE EXAM

If a question says the model overfits, the levers are max_depth down, eta down, subsample down, gamma up.

Linear Learner

- Linear and logistic models for classification and regression.
- Trains **many models in parallel** with different hyperparameters, and keeps the best on validation.
- Extremely fast and scalable. Very small model artefact.
- **Interpretable**: the coefficients are the explanation. That matters after module 12.
- **Needs feature scaling**, unlike XGBoost. It is a gradient-based method.

> ON THE EXAM
>
> The parallel-model search is its distinguishing feature, and it is what makes a weak-looking algorithm competitive.

XGBoost or Linear Learner?

XGBOOST, WHEN

The relationship is complex

- Interactions between features matter.
- Accuracy is the priority.
- Mixed types and missing values.
- SHAP attributions are explanation enough.

LINEAR LEARNER, WHEN

The relationship is simple

- Coefficients must be the explanation.
- Very high dimensional sparse data.
- Training speed or tiny artefact matters.
- A strong baseline before anything complex.

THE TIEBREAKER

AWS guidance: simple relationship, Linear Learner. Complex relationship, XGBoost. Interpretability can override both.

k-NN and Factorization Machines

- **k-NN** classifies or predicts from the k nearest training points. No real training phase.
- It is **distance based**, so scaling matters enormously, and it is slow at inference on large data.
- **Factorization Machines** handle **high-dimensional sparse** data specifically.
- The classic case is **recommendation**: a huge, mostly empty user-by-item matrix.
- Factorization Machines take pairwise interactions only. Not deeper ones.

> ON THE EXAM
>
> 'Sparse' is the Factorization Machines signal. Millions of users, millions of items, almost no ratings.

Gradient boosting is not only XGBoost

ALGORITHM	WHAT IT ADDS
LightGBM	Leaf-wise tree growth. Faster than XGBoost on very large tabular data
CatBoost	Handles categorical features natively, without one-hot encoding
TabTransformer	Attention applied to tabular data. Strong with many categoricals
AutoGluon-Tabular	Trains and ensembles several models automatically. Highest accuracy, most compute

ON THE EXAM

CatBoost is the one worth remembering: high-cardinality categoricals without the one-hot explosion from module 05.

SageMaker Autopilot

- Point it at a tabular dataset and a target column. It does the rest.
- It explores **feature engineering, algorithms and hyperparameters**, then ranks candidate models.
- **It generates notebooks** showing exactly what it tried. Not a black box.
- Right when a strong tabular baseline is needed fast, or nobody knows which algorithm fits.
- It costs far more compute than picking one algorithm, because it trains many.

> ON THE EXAM
>
> The generated notebooks are the underrated part: Autopilot doubles as a way to learn what a good pipeline for your data looks like.

Forecasting is not regression with a date column. It has its own algorithm.

DeepAR

- A recurrent neural network for forecasting one-dimensional time series.
- **It trains across many related series at once**, learning patterns they share. That is the whole point.
- So a new product with three weeks of history borrows seasonality from similar products.
- On a **single** time series, classical methods like ARIMA or ETS may well beat it.
- Works best with more than roughly **300 observations** across the training series.

> ON THE EXAM
>
> One series, use ARIMA. Hundreds or thousands of related series, use DeepAR. That contrast appears almost verbatim.

Data format and settings

- **JSON Lines**, one series per line, with a start timestamp and a target array.
- **prediction_length** sets how many time steps ahead it forecasts.
- **context_length** sets how much history it looks back over per prediction.
- It can take **categorical features** per series, such as product category or region.
- It produces **probabilistic forecasts**, giving quantiles rather than a single number.

> ON THE EXAM
>
> Probabilistic output is often the real reason to choose it. A demand forecast with a confidence band is far more useful for stock decisions.

These predate foundation models, and the exam still asks about them.

BlazingText

- Two modes: **Word2Vec** for word embeddings, and **text classification**.
- Extremely fast. It processes millions of words per second, and scales to very large corpora.
- Supervised mode classifies documents. Unsupervised mode produces word vectors.
- Word vectors here are word-level, not the contextual embeddings from module 09.
- Right when you need cheap classification at enormous volume.

> ON THE EXAM
>
> Speed and cost are the argument, not quality. A modern embedding model is better; BlazingText is far cheaper at scale.

Sequence-to-Sequence, Object2Vec, and the topic pair

- **Sequence-to-Sequence:** input sequence to output sequence. Machine translation, summarisation.
- **Object2Vec:** learns embeddings for **pairs of objects**, preserving their relationship.
- Object2Vec suits product search, item matching and recommendation from pair data.
- **LDA** and **Neural Topic Model** both find topics in a document collection, unsupervised.
- Sequence-to-Sequence is the one foundation models have most thoroughly replaced.

> ON THE EXAM
>
> Object2Vec is the odd one out: it is supervised on pairs, but the labels often come from natural structure rather than human annotation.

Reinforcement learning on SageMaker AI

- Learning by **trial and reward** rather than from labelled examples.
- Suits sequential decision problems: robotics, control systems, resource allocation.
- It needs an **environment to interact with**, usually a simulator. That is the real barrier.
- **RLHF** from module 11 is the form you are far more likely to meet in practice.
- Recognise it as a category. It is rarely the right answer on this exam.

> ON THE EXAM
>
> If an option offers reinforcement learning and the scenario has no environment or reward signal, it is a distractor.

LDA or Neural Topic Model?

LDA

Classical, and lighter

- Probabilistic, well understood, fast.
- Topics are readable word distributions.
- Works well on smaller collections.
- CPU is enough.

NEURAL TOPIC MODEL

Neural, and richer

- Neural variational inference.
- Handles larger and messier corpora.
- Can capture subtler topic structure.
- Benefits from GPU.

THE TIEBREAKER

Both are unsupervised and both output topics. If a question does not distinguish them on scale or infrastructure, either is defensible.

No labels means a different question: **what structure is in this data?**

K-Means and PCA

- **K-Means** partitions data into k clusters. You must choose k in advance.
- Customer segmentation is the standard case. Groups emerge; naming them is your job.
- Distance based, so **scaling matters**. An unscaled column will dominate the clustering.
- **PCA** reduces dimensions into uncorrelated components, keeping most of the variance.
- PCA components are **not interpretable**, from module 05. Regulated work rules it out.

> ON THE EXAM
>
> PCA is often a preprocessing step rather than an answer. Reduce dimensions, then train something else on the result.

Choosing k

- K-Means cannot tell you how many clusters exist. You assert it.
- **Elbow method:** plot within-cluster distance against k, and look for where it stops improving sharply.
- **Silhouette score:** measures how well each point fits its cluster against the next nearest.
- Often the business decides. Four marketing segments because there is budget for four campaigns.
- SageMaker AI K-Means can evaluate several values of k in one job.

> ON THE EXAM
>
> An unsupervised result has no accuracy score. Whether the clusters are useful is a judgement, not a measurement.

Random Cut Forest and IP Insights

- **Random Cut Forest** gives each data point an **anomaly score**. Unsupervised.
- It works on **streaming data**, which is what separates it from a batch anomaly method.
- Fraud detection, network monitoring, sensor faults, unusual metric behaviour.
- **IP Insights** learns usage patterns for **IPv4 addresses and entities**, such as user accounts.
- It flags an account logging in from an IP that does not fit its history. Purpose-built for that.

> ON THE EXAM
>
> Both are anomaly detectors and they are not interchangeable. IP addresses plus accounts is IP Insights; anything else is RCF.

Turning an anomaly score into an alert

- Random Cut Forest returns a **score**, not a verdict. Something must decide the threshold.
- A common rule: flag scores more than three standard deviations above the recent mean.
- **The threshold is the alert-fatigue dial.** Too low and nobody reads the alerts.
- Thresholds usually need to vary by metric, which is why per-metric static rules do not scale.
- Same shape as A2I's confidence threshold in module 07: a score plus a business decision.

> ON THE EXAM
>
> An anomaly detector that fires constantly is worse than none, because it trains people to ignore the channel.

The three vision algorithms

- **Image Classification:** what is this a picture of? One label per image.
- **Object Detection:** what is in it and **where**? Bounding boxes with labels.
- **Semantic Segmentation:** which class does **every pixel** belong to?
- Increasing precision, increasing labelling cost. Segmentation masks are the most expensive labels there are.
- All three support transfer learning from pre-trained weights, which is module 12's JumpStart argument.

> ON THE EXAM
>
> The question tells you which: 'is there a defect' is classification, 'where is it' is detection, 'how large is it' is segmentation.

What the vision algorithms need from you

- **Classification** needs one label per image. Cheapest labelling by a wide margin.
- **Detection** needs bounding boxes with class labels. Manifest files carry the coordinates.
- **Segmentation** needs a mask image per training image. Tens of seconds of human work each.
- All three benefit strongly from **transfer learning**: start from pre-trained weights, not random ones.
- **GPU instances** are effectively required. These are convolutional networks.

> ON THE EXAM
>
> Labelling cost, not algorithm choice, is usually what decides which of the three a project can afford.

Problem to algorithm

WHAT YOU ARE DOING	ALGORITHM
Tabular prediction, accuracy matters	XGBoost
Tabular, coefficients must explain it	Linear Learner
Sparse user-item recommendation	Factorization Machines
Forecasting many related time series	DeepAR
Cheap text classification at huge scale	BlazingText
Topics in a document set, no labels	LDA or Neural Topic Model
Embeddings from pairs of objects	Object2Vec
Customer segmentation, no labels	K-Means
Too many features, reduce them	PCA
Anomaly score, streaming, no labels	Random Cut Forest
Unusual login by IP and account	IP Insights
What is in this image, and where	Object Detection

Input formats and instance types

CONSIDERATION	WHAT TO KNOW
CSV	Accepted widely. First column is the label. No header row
RecordIO-protobuf	Preferred by most. Enables Pipe mode, from module 03
JSON Lines	DeepAR uses it, one series per line
GPU instances	Vision algorithms, NTM, and DeepAR benefit
CPU instances	Linear Learner, K-Means, PCA, RCF are usually fine
XGBoost	CPU by default. tree_method gpu_hist trains on GPU

ON THE EXAM

Choosing GPU for an algorithm that cannot use it is a real and expensive mistake. Not everything parallelises that way.

Channels, and what each is for

CHANNEL	PURPOSE
`train`	The training data. Always required
`validation`	Held-out data for early stopping and model selection
`test`	Optional. Some algorithms report a final score on it
`model`	For incremental training: a previously trained model to continue from
`Content type`	Set per channel, e.g. text/csv or application/x-recordio-protobuf

ON THE EXAM

Without a validation channel, early stopping cannot work and the algorithm has no basis for choosing between candidates.

Incremental training

- Continue training from a **previously trained model artefact** rather than random initialisation.
- Cheaper and faster than retraining from zero, and it keeps what the model already learned.
- Supported by the vision algorithms and by a subset of the others. Check per algorithm.
- It carries the same **catastrophic forgetting** risk as module 11 if new data is narrow.
- The usual retraining pattern: new data plus a sample of the old, not new data alone.

> ON THE EXAM
>
> This is the mechanism behind cheap scheduled retraining. Module 28 puts it in a pipeline.

What a training job actually produces

ARTEFACT	WHERE IT GOES
`model.tar.gz`	The trained model, written to your S3 output path
`Training metrics`	Emitted to CloudWatch, and visible in the console
`Objective metric`	The single value automatic model tuning optimises. Module 15
`Checkpoints`	Written during training if configured. Essential for Spot
`Job metadata`	Hyperparameters, data locations, image URI. Your reproducibility record

ON THE EXAM

The job metadata is the reproducibility record from module 11. It records exactly what produced this artefact.

Metrics algorithms emit, and what they tell you

- Most emit a **training** metric and a **validation** metric per epoch or per round.
- **Both falling** is healthy. Keep going.
- **Training falling, validation rising** is overfitting. Stop, and take the earlier checkpoint.
- **Both flat and high** is underfitting. More capacity, more features, or more rounds.
- The metric name matters: **validation:auc**, **validation:rmse**, **validation:f1** and so on.

> ON THE EXAM
>
> Three curve shapes, three diagnoses. This is the same picture as module 11's fine-tuning loss curves.

Built-in container or script mode?

BUILT-IN, WHEN

Configuration is enough

- The standard implementation fits.
- You want zero code to maintain.
- Distributed training out of the box.
- Hyperparameters give enough control.

SCRIPT MODE, WHEN

You need to change behaviour

- A custom loss or metric is required.
- Preprocessing must live inside training.
- A specific library version is needed.
- The framework container still handles infrastructure.

THE TIEBREAKER

XGBoost exists both ways: as a built-in algorithm and as a framework container you write code against. The choice is control, not capability.

Where built-in algorithms still beat foundation models

- **Tabular prediction.** Not close. Gradient boosting wins decisively.
- **Latency.** Milliseconds against hundreds of milliseconds, from module 12.
- **Cost at volume.** No per-token charge, and a tiny model artefact.
- **Explainability.** SHAP on a tree model satisfies regulators. Narrative does not.
- **Determinism.** The same input gives the same output, every time. That matters more than it sounds.

> ON THE EXAM
>
> Determinism is the quiet one. A pricing or credit decision that varies between identical requests is very hard to defend.

Cost shape of a built-in algorithm

COMPONENT	BEHAVIOUR
Training	Per instance-second, for the duration of the job only
Spot training	Up to a large discount, with interruption. Needs checkpoints. Module 14
Hyperparameter tuning	Training cost multiplied by the number of jobs. Module 15
Inference endpoint	Per instance-hour, continuously, until you delete it
Batch transform	Per instance-second, for the job only. No idle cost
Autopilot	Many training jobs, so many multiples of a single one

ON THE EXAM

Training is a bounded cost. An endpoint is an unbounded one. That asymmetry drives most ML cost surprises.

When a built-in is the wrong tool

- The task is a language task. Bedrock, from module 12, not BlazingText.
- You need a specific published architecture. That is script mode or JumpStart.
- The data is images and you have few labels. JumpStart transfer learning beats training from scratch.
- Custom loss functions, multi-task learning, or anything research-shaped.
- Built-ins cover the common commercial cases extremely well, and they are not a general ML platform.

> ON THE EXAM
>
> The honest framing: built-in algorithms are a well-chosen set of defaults, not the whole of machine learning.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Training error near zero, test error poor	Overfitting. max_depth or num_round too high
K-Means clusters dominated by one feature	That feature is unscaled and larger in magnitude
DeepAR forecast worse than a simple average	One series only, or too few observations
Model rejects the training file	Header row present, or the label is not the first column
GPU instance no faster than CPU	The algorithm does not use GPU acceleration
Recommendation model will not fit in memory	Dense encoding of sparse data. Use Factorization Machines
Anomaly detector flags almost everything	The score threshold was never tuned

ON THE EXAM

Half of these are module 05 problems appearing again: scaling, encoding and splitting decide algorithm behaviour.

Name the problem shape first. The algorithm follows from it almost mechanically.

A telecoms company predicts monthly customer churn from 60 fields: contract type, tenure, monthly charges, support call counts, region and payment method. Some fields have missing values.

They have 2 million labelled rows. Accuracy is the priority. A data science team will own the model, and SHAP attributions satisfy their compliance team.

Which algorithm should they use?

A K-Means, to segment customers into churn-risk groups

B Linear Learner, for a fast and interpretable baseline

C XGBoost, tuned for classification

D Factorization Machines, to handle the categorical fields

◆ PAUSE AND DECIDE

Which algorithm should they use?

A K-Means, to segment customers into churn-risk groups

Unsupervised, and 2 million labelled rows are available. Clustering discards the labels and does not predict churn for an individual.

B Linear Learner, for a fast and interpretable baseline

A good baseline, and the scenario says accuracy is the priority and SHAP is sufficient. That removes the reason to accept a weaker model.

C XGBoost, tuned for classification

Tabular, labelled, mixed types, missing values, accuracy first. Every stated fact points here, and SHAP covers the compliance requirement.

D Factorization Machines, to handle the categorical fields

Built for high-dimensional sparse data such as user-item matrices. Sixty dense fields is not that problem.

THE SIGNAL

Note what the SHAP sentence does: it removes interpretability as a constraint, which is the only thing that would favour Linear Learner.

> A grocery chain forecasts weekly demand for 8,000 products across 200 stores. Products share strong seasonal patterns, and around 400 lines are new each quarter with under a month of history.
>
> The planning team needs a range rather than a single number, so they can set safety stock.

Which algorithm fits?

A. DeepAR, trained across all product-store series together

B. Linear Learner with the week number as a feature

C. A separate ARIMA model per product-store combination

D. Random Cut Forest on the demand history

◆ PAUSE AND DECIDE

Which algorithm fits?

A DeepAR, trained across all product-store series together

Many related series with shared seasonality, cold-start products, and a probabilistic forecast required. All three are exactly what DeepAR provides.

B Linear Learner with the week number as a feature

Treats forecasting as regression with a date column. It cannot represent autocorrelation, and gives a point estimate with no range.

C A separate ARIMA model per product-store combination

1.6 million models to fit and maintain, and each one is blind to the others. The 400 new lines have almost no history to fit on.

D Random Cut Forest on the demand history

An anomaly detector. It scores how unusual a point is; it does not forecast future values at all.

THE SIGNAL

The cold-start detail is the giveaway. Only an algorithm trained across related series can forecast a product with three weeks of history.

> A platform team streams thousands of infrastructure metrics per minute. They want to be alerted when a metric behaves unusually, without defining a threshold for each one by hand.
>
> There are no labelled examples of incidents. The behaviour of normal varies by time of day and day of week.

Which algorithm should they use?

A XGBoost, trained to classify each metric reading as normal or abnormal

B K-Means, clustering readings and alerting on the smallest cluster

C IP Insights, to learn normal usage patterns

D Random Cut Forest, scoring each reading for anomaly

◆ PAUSE AND DECIDE

Which algorithm should they use?

A XGBoost, trained to classify each metric reading as normal or abnormal

Supervised, and the scenario states there are no labelled incidents. There is nothing to train the classifier on.

B K-Means, clustering readings and alerting on the smallest cluster

Unsupervised, and clustering is not anomaly scoring. A small cluster may be a legitimate rare pattern, and there is no per-reading score.

C IP Insights, to learn normal usage patterns

Purpose-built for IP addresses paired with entities such as accounts. Infrastructure metrics are neither.

D Random Cut Forest, scoring each reading for anomaly

Unsupervised anomaly scoring designed for streaming data, giving a per-reading score you can threshold once rather than per metric.

THE SIGNAL

'No labelled examples' eliminates every supervised algorithm in one phrase. Listen for it; it does a lot of work.

A security team wants to flag when a corporate user account authenticates from an IP address that does not fit that account's historical pattern. They have two years of successful login records, each holding an account identifier and an IPv4 address.

An engineer proposes Object2Vec, reasoning that it learns embeddings for pairs of objects and each login is an account-address pair.

How should you respond?

A Agree. Object2Vec is designed for pairwise relationships, which this is

B Redirect to IP Insights, which is purpose-built for account and IPv4 pairs

C Redirect to Factorization Machines, for the sparse account-address matrix

D Redirect to XGBoost, trained on the login records as labelled data

◆ PAUSE AND DECIDE

How should you respond?

A. Agree. Object2Vec is designed for pairwise relationships, which this is

The reasoning is sound and the conclusion is not. Object2Vec is general-purpose; a purpose-built algorithm for this exact pair type exists.

B. Redirect to IP Insights, which is purpose-built for account and IPv4 pairs

It learns usage patterns for IPv4 addresses paired with entities and flags logins that do not fit history. The scenario is its documented use case.

C. Redirect to Factorization Machines, for the sparse account-address matrix

Built for recommendation from sparse interaction data. It predicts affinity, not anomaly, so it answers a different question.

D. Redirect to XGBoost, trained on the login records as labelled data

The records are all successful logins. There are no negative examples, so there is nothing to classify against.

THE SIGNAL

The general-purpose algorithm would work. The purpose-built one works better and needs less effort. That is the whole comparison.

What to take from this module

- **Data type, labels, output.** Three questions eliminate most of the list.
- **XGBoost is the tabular default.** Linear Learner when coefficients must explain it.
- **DeepAR for many related series.** One series alone favours ARIMA.
- **RCF for anomaly scores. IP Insights only for IP-and-entity pairs.**
- **'No labelled examples'** is the phrase that eliminates every supervised algorithm at once.

> ON THE EXAM
>
> Module 14 covers how a training job actually runs: script mode, containers, distributed training and Spot.